

Scratch Jr Coding Framework

Comprehensive K-2 Lesson Plan: Catch the Star Game

Target Grade Level:	Kindergarten to Grade 2 (Ages 5–8)
Core Concepts Covered:	Sequencing, Event Handlers, Forever Loops, Debugging
Primary Application:	Scratch Jr (Tablet / Mobile Interface)
Lesson Duration:	60 to 90 Minutes

1. Introduction & Contextual Anchor

Welcome to the **Catch the Star** block-based coding instructional module! This curriculum is tailored for primary school teachers, ICT coordinators, and educators designed to introduce abstract computational thinking paradigms through relatable cultural touchstones and localized narratives.

The pedagogical strategy relies heavily on **contextual framing**. By mapping logic onto real-world events that children live through, we lower cognitive load and increase engagement. This lesson bridges the magical, sensory experience of looking at a night sky during traditional celebrations with active digital creation.

Cultural Anchor & Hook: Diwali Nights

Have you ever stepped onto your roof during a crisp Diwali night and observed the brilliant stars twinkling high above? Imagine the wonder if you could reach into the sky and catch one! In this module, students transition from passive dreamers to active digital architects, building an interactive game that makes catching a star a pixelated reality.

Throughout this guide, students will learn to customize their workspace, animate avatars, command interactive rules, and systematically track down programmatic errors. Let's begin by introducing the digital laboratory workspace.

2. Meeting the Interface (The Digital Workspace)

Before writing software, young learners must understand the taxonomy of their workspace. The Scratch Jr interface is divided into functional zones that closely mirror physical theaters or play areas:

01. The Canvas

The Canvas serves as the physical stage where the software executes. This is where the game comes alive! All active graphic assets, called *sprites*, move, collide, transform, and execute their assigned computational scripts within this coordinates frame.

02. The Background Library

This repository allows users to configure the structural environment of their game. For our core project, students will navigate here to choose a beautiful night sky or deep outer space backdrop, establishing the mood for their nighttime catch game.

03. The Sprite Palette

The Character / Sprite Palette functions as a backstage casting house. This is where characters are selected, created, or modified. Learners will choose assets like a high-contrast twinkling star, an interactive catcher, or an adventurous astronaut to cast into their digital production.

3. Core Concept: Sequencing

In computer science, a **sequence** refers to the precise, chronological order in which individual commands or statements are processed by a computer system. Computers do not possess intuition; they execute instructions blindly from top to bottom, or left to right. If a single step is misplaced or omitted, the broader program will experience catastrophic logic collapse.

The Real-World Analogy: Making Nimbu Pani

To make this clear to early learners, we map algorithmic sequencing directly onto the physical steps required to prepare *Nimbu Pani* (traditional Indian lemonade). The sequence must follow a strict logical order:

Step ID	Action Description	Computational Thinking Equivalence
01	Pour pure water into a tall, clean glass and add a pinch of salt to construct the chemical base.	Initialization: Establishing the foundational state and environment variables before runtime.
02	Squeeze fresh lemon juice into the vessel. The greater the volume, the tangier the concentration!	Input Parameters: Passing quantitative variables into the system to alter internal attributes.
03	Mix in fine sugar and stir vigorously until fully dissolved.	Execution / Processing: Modifying structural states to yield the final desired output.

The Breakage Law of Logic:

If the sequence order changes, the program breaks! Consider what happens if you attempt to stir sugar and squeeze lemon before pouring water into the glass: you wind up with dry granules clinging to an empty vessel and sticky counter spills. The algorithmic output is ruined because the sequence violated its prerequisite rules.

4. Core Concept: Event Handlers (Triggering Actions)

An **Event** is an external occurrence or signal that a software application listens for. An **Event Handler** is the specific code block that wakes up and runs in direct response to that trigger. Without event handlers, software remains completely inert and unresponsive.

Example A: Clapping Starts the Song

Imagine walking into a dark room at a birthday gathering. Everyone stands perfectly silent. Suddenly, a designated leader claps their hands. Immediately, the entire group begins singing the birthday song. In this context, the physical act of clapping serves as the **Event Trigger**, while singing the song is the **Action** executed. In Scratch Jr, tapping the **Green Flag** block or touching the mobile screen functions exactly like that "clap"—it sends a broad signal across the system saying: *"Wake up, initialize parameters, and start running the game!"*

Example B: School Bell Starts Recess

Consider a classroom of students focusing intently on their schoolwork. The moment the high-pitched school bell rings, the children immediately drop their pencils and rush outside to play on the grass. The bell ringing is the event; running outside is the action.

When drafting our digital project, interactive touch represents this bell. Touching or clicking on the star sprite acts as our "school bell"—it registers a physical contact event and immediately triggers the contextual **catch action**.

5. Core Concept: Loops (Repetition Control Structures)

A **Loop** is a programming structure that instructs a computer to execute a set of code blocks repeatedly, either for a specified count or for an infinite duration based on specified conditional terms. Loops are essential for keeping programs efficient, eliminating the need to write identical lines of instructions over and over again.

Defining the Concept

"Loops repeat actions forever, creating patterns without an end-state!"

Cultural & Architectural Visualizations of Loops

The Urban Traffic Signal Loop:

Observe a municipal traffic intersection. The control system alternates the light signals in a rigid, repeating loop: **Red**, then **Yellow**, then **Green**, and then back to **Red**. This process repeats continuously all day and night without stopping. It uses a loop structure to maintain systemic order.

The Culinary Jalebi Loop:

Watch a sweetmaker prepare fresh *Jalebis* in a large frying vat. The chef doesn't drop batter in straight, disconnected strokes. Instead, they swirl their hand round and round in hot oil, forming repeating concentric circles without breaking the flow. Each loop adds to the geometry of the pastry.

6. Step-by-Step Sprite Configuration Guide

Now that our theoretical foundation is established, we will apply these concepts within the Scratch Jr platform to build our *Catch the Star* game. Follow these systematic instructions to cast and assemble the digital assets.

Step 1: Adding the Sprite Character Assets

Open your active project canvas. We must first declare two core actor assets in the Sprite Palette to drive our game mechanics:

1. **The Catcher Sprite:** Select an asset designed for horizontal movement. Educators can recommend selecting a mischievous monkey character, a nimble explorer, or an astronaut. This asset will move along the lower edge of the screen to catch falling items.
2. **The Target Star Sprite:** Choose a clear, distinct target icon, such as a high-contrast twinkling star or a ripe golden mango. Once spawned, drag this asset to the upper boundary of your digital stage to establish its starting position.

Design Variety & Personalization Note

Encourage your students to explore diverse Indian characters and regional cultural items within the asset menu. They can substitute an astronaut with a traditional character, or swap a basic yellow star for a mango to customize the project into something uniquely their own!

7. The Code Architecture (Script Drafting)

With our assets cast on the stage, we must give them functionality by assembly block scripts. Let's look at the exact step-by-step logic required for both sprites.

A. Scripting the Catcher Sprite

The primary objective for the Catcher is to move smoothly back and forth along the bottom of the screen as soon as the game begins.

Visual Icon	Block Group & Name	Functional Mechanism
[Green Flag]	Trigger Group: <i>Start on Green Flag</i>	Acts as our system initialization event. Tapping this flag wakes up the Catcher asset and prepares it to run its movement blocks.
[Right Arrow]	Motion Group: <i>Move Right (Value: 5)</i>	Commands the sprite to change its position 5 spatial units to the right along the horizontal axis.
[Left Arrow]	Motion Group: <i>Move Left (Value: 5)</i>	Commands the sprite to reverse direction and slide 5 spatial units back to the left.
[Loop Arrow]	Control Group: <i>Repeat Forever Loop</i>	Wraps completely around both the Move Right and Move Left blocks, forcing the sprite to slide back and forth across the screen continuously.

B. Scripting the Star Target Sprite

The Star sprite needs a dynamic script. It stays still until the player catches it, at which point it gives sensory feedback, disappears, and reappears somewhere else on the screen to keep the game going.

- **Trigger Event:** Choose the *Start on Tap* block (triggered when the player touches or collides with the sprite). This tells the system that a successful "catch" has occurred.
- **Audio Feedback:** Snap a *Play Pop Sound* block immediately after the trigger to give the player instant auditory feedback.
- **Visual Disappearance:** Attach a *Hide* block. The star disappears from view, simulating that it was successfully caught and pocketed.
- **Temporal Delay:** Insert a *Wait Block* set to exactly $t = 2.0$ seconds. This gives the player a brief pause to reposition themselves.

- **Reposition & Display:** End the sequence with a *Show at Random Place* command block. This teleports the star to a new, unpredictable spot on the canvas and makes it visible again, ready for the next catch!

8. Differentiated Micro-Challenges

To accommodate different learning speeds and encourage student agency, introduce these three micro-challenges once the base project is running. These tasks prompt students to tinker with variables and try out advanced block groups:

Challenge 01: Rotational Kinetic Spin

Make the star spin around rapidly the moment the catcher touches it! Students must find the orange **Rotate Block** and insert it right after the touch trigger event to add rotational animation.

Challenge 02: Dynamic Scale Growth

Program the star to grow larger every time it respawns on screen. Students will need to use the purple **Grow Block** within the visibility script, introducing them to dynamic asset scaling.

Challenge 03: Workspace Theme Swap

Completely re-theme the game project! Swap out the standard astronaut for a clever monkey character, or change the target star into a sweet mango, testing the student's ability to navigate and manage the asset libraries.

9. Debugging Made Easy

In computer engineering, a **bug** is an error, flaw, or fault in a software program that causes it to produce an incorrect or unexpected behavior. **Debugging** is the systematic process of finding and resolving these issues. For young kids, debugging is a fantastic way to build resilience and logical troubleshooting skills.

Common Student Bugs & Solutions:

- **The Static Sprite Bug:** *The Catcher won't move at all when the green flag is clicked!*
Solution Check: Look at the script area and ensure the yellow **Start on Green Flag** block is physically snapped to the front of the blue movement blocks. If it's floating nearby without touching, the command chain won't execute!
- **The Vanishing Star Bug:** *The Star vanishes after the first touch and never returns to the screen.*
Solution Check: Check the tail end of the star's script. Students often add a **Hide** block but forget to attach the final purple **Show** block at the very end. The code is running, but the asset is stuck in an invisible state!

10. Lesson Recap & Formative Assessment

Wrap up the session by summarizing the three core computational concepts explored today. Use these definitions for a quick verbal review or quiz:

- 1. Sequence:** Step-by-step instructions that must follow a strict, correct order—just like preparing *Nimbu Pani*. If you scramble the steps, the program breaks!
- 2. Events:** A specific trigger action (like a finger tap or mouse click) that kicks off a code script—similar to a loud school bell starting recess or hands clapping to start a song.
- 3. Loops:** Control blocks that repeat actions continuously without stopping—like a traffic light cycling through its colors or a jalebi swirling in hot frying oil.

Take-Home Activity Assignment:

Ask students to open their home play sandbox workspace and design an independent variant game using at least two custom characters from the asset library. The project must include one continuous motion loop and use a screen-touch event trigger. Parents are encouraged to test and play their child's creation!